

TorchSigGUI 0.0.1 Release Preparation

Change report for the developer

2026-09-16

Repository: torchsig-gui (target `https://github.com/TorchDSP/torchsig-gui`)

Baseline: commit `8bb328b` “initial commit”

Purpose. Prepare TorchSigGUI for a first public **0.0.1 release on GitHub**. PyPI publishing is explicitly out of scope, so users install from GitHub (`pip install git+https://...@v0.0.1`, or clone + `pip install .`). The TorchDSP/torchsig repository was used as the reference for project conventions: issue/PR templates, CI workflows, Makefile, security policy, and `vX.Y.Z` tags.

State of the work. All changes are in the working tree only. **Nothing has been staged, committed, tagged, or pushed.** Reviewing, committing, and tagging are left to you.

Contents

- 1. Summary of changes
- 2. Decisions and constraints from the project owner
- 3. P0: Release blockers (bugs and incorrect metadata)
- 4. P1: Repository hygiene, CI and linting
- 5. README: local vs. remote (SSH tunnel) usage
- 6. P2: Makefile, security policy, tests folder rename
- 7. Verification performed
- 8. Complete file inventory
- 9. Items needing your attention
- 10. Suggested commit and release procedure

1. Summary of changes

Area	What changed	Why it matters
Versioning	Version set to <code>0.0.1</code> in <code>torchsiggui/__init__.py</code> , <code>package.json</code> , <code>package-lock.json</code> .	Previously <code>0.0.0a1</code> (Python) vs <code>0.0.1-alpha</code> (npm).
Packaging	Fixed <code>pyproject.toml</code> : description, license, license-files, package data, dev extras, pinned TorchSig, URLs, classifiers, coverage target, ruff config.	A non-editable install (<code>pip install git+...</code>) previously shipped <i>without the frontend and SQL file</i> .

Area	What changed	Why it matters
Server bugs	Awaited the stale-worker DB deletes; handled dead PIDs in <code>main.py</code> .	Stale worker records were never removed; a crashed worker could crash startup.
Data location	Datasets moved from “next to the package” to <code>~/.cache/torchsiggui</code> (overridable).	For normal installs, data was written into, and <code>rmtree</code> ’d from, <code>site-packages</code> .
Frontend build	Build now outputs directly to <code>torchsiggui/webbuild</code> with a fixed build ID.	<code>npm run build</code> previously wrote to <code>./webbuild</code> , and output was not reproducible.
Frontend code	WebSocket URL derived from the page host; 31 ESLint errors fixed (types, <code>const</code> , store creation).	Needed for SSH tunnels on a different local port; lint must pass in CI.
Linting	ESLint (Next.js config) and Ruff configured; unused imports and variables removed.	<code>npm run lint</code> previously failed (ESLint not installed, no config).
GitHub	CI workflow, release workflow, bug/feature issue templates, PR template, <code>.gitattributes</code> .	Automated checks, and GitHub Releases replace PyPI as the distribution path.
Docs	README (install URLs, TorchSig pin, data folder, dev setup, local vs remote usage), <code>.env.example</code> , <code>LICENSE</code> copyright, <code>SECURITY.md</code> .	README pointed to a redacted URL and a missing <code>.env.example</code> .
Tooling	<code>Makefile</code> ; <code>testing/</code> renamed to <code>tests/</code> .	Matches TorchSig conventions.

2. Decisions and constraints from the project owner

- **No PyPI.** Distribution is via GitHub tags and Releases only.
- **Repository URL:** `https://github.com/TorchDSP/torchsig-gui`.
- **Copyright holder:** TorchSig.
- **TorchSig dependency:** locked to exactly `torchsig==2.2.0`.
- **No git commands** were to be run on the owner’s behalf. *Disclosure:* two read-only commands were run by mistake: `git status --short` during baseline testing, and `git diff --stat` with its output discarded. Neither modified the repository, the index, or history. The folder rename was done with plain `mv`, not `git mv`.
- **Declined:** `CHANGELOG.md`; README “polish” (badges, screenshot, citation, relaxing prerequisites).
- **Not requested, so skipped:** `CONTRIBUTING.md`, Dockerfile, CI coverage upload.
- **Accepted:** all P0 fixes; `.github/`; ESLint and Ruff; `.gitattributes`; `Makefile`; `SECURITY.md`; rename to `tests/`; README section on local vs remote use.

3. P0: Release blockers

3.1 Consistent version number

Files: `torchsiggui/__init__.py`, `package.json`, `package-lock.json`

Before: Python reported `__version__ = "0.0.0a1"`, while npm reported `"0.0.1-alpha"`. `torchsiggui --version` and the wheel filename come from the Python value, so the release would have shipped as `0.0.0a1`.

After: both are `0.0.1`, including the lockfile's two root `version` fields. The release tag should be `v0.0.1`, following TorchSig's `vX.Y.Z` convention. The new release workflow enforces that the tag matches `__version__`.

3.2 `pyproject.toml` metadata and packaging

File: `pyproject.toml`

Setting	Before	After, and reason
<code>build-system.requires</code>	<code>["setuptools>=61", "build"]</code>	<code>["setuptools>=77"]</code> . The SPDX string form of <code>license</code> requires setuptools 77+. <code>build</code> is a build frontend, not a backend requirement.
<code>description</code>	"Signal Processing Machine Learning Toolkit" (copied from TorchSig)	"Graphical interface for building TorchSig datasets".
<code>license</code>	<code>"MIT AND Apache-2.0 AND BSD-2-Clause AND BSD-3-Clause"</code>	<code>"MIT"</code> . No third-party Python code is vendored; dependencies are only imported, so their licenses are not part of this package's license expression.
<code>license-files</code>	<code>["LICENSE.md", "licenses/LICENSE*"]</code>	<code>["LICENSE"]</code> . <code>LICENSE.md</code> does not exist; the file is <code>LICENSE</code> .
<code>keywords</code>	signal processing, machine learning	Added <code>torchsig</code> , <code>gui</code> .
<code>classifiers</code>	Python, Python 3	Added Development Status 3 (Alpha), Framework FastAPI, Intended Audience Science/Research, OS POSIX Linux, Python 3 Only, Topic Scientific/Engineering.
Runtime <code>dependencies</code>	<code>fastapi</code> and <code>torchsig</code> unpinned; <code>pytest</code> , <code>pytest_asyncio</code> , <code>pytest-timeout</code> , <code>pytest-cov</code> , <code>httpx2</code> listed as runtime deps	<code>fastapi>=0.141</code> : the code uses <code>app.frontend()</code> , confirmed present in 0.141.1. <code>torchsig==2.2.0</code> : the code imports TorchSig internals that can change between versions. Test tools moved out of runtime deps.
<code>[project.optional-dependencies]</code>	none	New <code>dev</code> extra: the <code>pytest</code> packages, <code>httpx2</code> , and <code>ruff</code> . Install with <code>pip install -e ".[dev]"</code> .

Setting	Before	After, and reason
<code>[project.urls]</code>	commented-out TODO	Homepage, Repository, Issues (GitHub), TorchSig (torchsig.com).
<code>packages.find</code>	<code>where = ["."]</code>	Added <code>include = ["torchsiggui*"]</code> so only the package is discovered, not <code>tests</code> , <code>node_modules</code> , etc.
<code>package-data</code>	<code>files = ["*.sql"]</code> and <code>webbuild = ["*"]</code>	<code>torchsiggui = ["files/*.sql", "webbuild/**/*"]</code> . Bug fix: the keys must be package names. <code>files</code> and <code>webbuild</code> are not packages, so a wheel shipped without <code>database_sql.sql</code> and without the frontend. This went unnoticed only because editable installs read straight from the source tree.
<code>pytest addopts</code>	<code>--cov=torchsig</code>	<code>--cov=torchsiggui</code> . Coverage was measuring the TorchSig dependency instead of this project.
<code>testpaths</code>	<code>["testing"]</code>	<code>["tests"]</code> (see §6.3).
<code>[tool.ruff]</code>	none	New; see §4.3.

Note on `httpx2` : it was kept as-is because it is what is installed in the working venv and the tests pass with it. Please confirm it is the intended package name (FastAPI's `TestClient` traditionally depends on `httpx`).

3.3 Server lifespan bugs

File: `torchsiggui/main.py`

Bug 1: un-awaited coroutine. In both the startup and shutdown sections of `startup_shutdown()`, stale worker records were removed with `run_query(queries.delete_worker_entry, ...)` *without* `await`. `run_query` is `async`, so this only created a coroutine object that never ran. Stale records were never deleted, and Python emitted “coroutine was never awaited” warnings. Stale records affect `get_worker_count`, which decides whether the dataset folder is deleted at shutdown.

Bug 2: crash on a dead PID. `psutil.Process(worker['pid'])` raises `psutil.NoSuchProcess` when that PID no longer exists, which is exactly the “crashed worker” case the loop is meant to clean up. The server would fail during startup or shutdown instead of removing the record.

Fix. The duplicated loop was extracted into one helper that is awaited in both places:

```
# Removes the database records of workers that are no longer running
async def remove_crashed_workers():
    worker_map = await get_worker_info()
    for worker in worker_map:
        try:
            process = psutil.Process(worker['pid'])
            crashed = not process.is_running() or process.create_time() != worker['created']
        except psutil.NoSuchProcess:
            crashed = True
        if crashed:
            await run_query(queries.delete_worker_entry, process_id=worker['pid'])
```

Startup and shutdown now call `await remove_crashed_workers()`. The PID-reuse check (comparing `create_time`) is preserved.

Separately, the Ruff cleanup (§4.3) removed the now-unused `MODULE_LOCK_FILE` and `FileLock` imports. The commented-out locking code is still there for future multi-worker support and would need those imports restored if enabled.

3.4 Dataset storage location

File: `torchsiggui/files/file_io.py`

Before:

```
MODULE_PARENT_FOLDER = MODULE_FOLDER.parent
MODULE_LOCK_FILE = MODULE_PARENT_FOLDER / 'workers.lock'
DATASET_FOLDER = MODULE_PARENT_FOLDER / 'datasets'
```

`MODULE_FOLDER` is the installed `torchsiggui` package directory. With an editable install, its parent is the repository root, which works. With a normal install (the `pip install git+...` path the README now documents), its parent is `site-packages/`. The server would create `site-packages/datasets` (often not writable) and, at shutdown, `rmtree` it.

After:

```
# - Uses TORCHSIGGUI_DATA_DIR if set, otherwise the user cache folder, so data is never written into
  the install location
DATA_FOLDER = Path(environ.get('TORCHSIGGUI_DATA_DIR') or Path(environ.get('XDG_CACHE_HOME') or
    Path.home() / '.cache') / 'torchsiggui')
MODULE_LOCK_FILE = DATA_FOLDER / 'workers.lock'
DATASET_FOLDER = DATA_FOLDER / 'datasets'
```

- **Default:** `~/.cache/torchsiggui/datasets`, or `$XDG_CACHE_HOME/torchsiggui/datasets` if that variable is set.
- **Override:** the `TORCHSIGGUI_DATA_DIR` environment variable. It is read from the real process environment, *not* from `.env`; only `AppConfig` reads `.env`.
- No new dependency (such as `platformdirs`) was added.
- Only `datasets/` is removed at shutdown, never the whole cache folder.
- `MODULE_PARENT_FOLDER` was removed; nothing else referenced it. All other modules and tests still import `DATASET_FOLDER` and `DATABASE` unchanged.

3.5 README, `.env.example`, LICENSE, Node version

Files: `README.md`, `.env.example` (new), `LICENSE`, `package.json`

- **Clone URL:** replaced the placeholder `https://<redacted-host>/torchsig/torchsig-gui.git` with `https://github.com/TorchDSP/torchsig-gui.git`. Added the direct install option `pip install git+https://github.com/TorchDSP/torchsig-gui.git@v0.0.1`. Changed `pip install -e .` to `pip install .` for users; the editable install is now in the Development section.
- **Prerequisites:** added “TorchSig 2.2.0 (installed automatically)”. The other prerequisites were intentionally left unchanged.
- **`.env` wording:** now says the `.env` file is read from the folder where the server is *started* (pydantic reads it relative to the working directory), with a link to `.env.example`.
- **Data folder:** documented the default location and `TORCHSIGGUI_DATA_DIR`.
- **Development setup:** new “Setup” subsection: Node.js ≥ 20.9, `pip install -e ".[dev]"`, `npm install`.
- **`.env.example` (new):** the README referenced it but it was missing. It contains `TORCHSIGGUI_PORT=8000` with comments, and `.gitignore` already whitelists it. `TORCHSIGGUI_DEV_MODE` was left out on purpose because dev mode is an internal flag.
- **LICENSE:** `Copyright (c) 2026–2026 TorchSigGUI` → `Copyright (c) 2026 TorchSig`.
- **`package.json`:** added `"engines": { "node": ">=20.9.0" }`, the minimum required by Next.js 16.3.4.

Not changed: `NOTICE`, `THIRD_PARTY_LICENSES`, and `licenses/` are untouched, and they are no longer listed in the package metadata. Such files are only needed for redistributed code. The Python dependencies they list are not redistributed, but the built `webbuild/` JavaScript bundle *does* redistribute npm packages (React, Bootstrap, Redux, etc.). A decision is still needed; see §9.

4. P1: Repository hygiene, CI and linting

4.1 Frontend build output and reproducibility

Files: `package.json`, `next.config.ts`, `torchsiggui/webbuild/**` (regenerated)

Problem found while setting up CI.

- **Wrong output folder.** The build script was `NEXT_STATIC_BUILD=webbuild next build`, and `next.config.ts` uses that value as `distDir`, relative to the project root. The build therefore went to `./webbuild`, *not* `torchsiggui/webbuild`, which is the folder the server serves and the wheel packages. The committed bundle must have been copied there by hand.
- **Random build ID.** Next.js assigns a random build ID on every build (e.g. `_next/static/tMHizx5hQb_ydNult2RoH/`), so two builds of identical source always differed. A CI “is the committed build up to date?” check would always fail.

Fix:

```
// package.json
"build": "NEXT_STATIC_BUILD=torchsiggui/webbuild next build"

// next.config.ts
// Uses a fixed build ID so rebuilding unchanged source reproduces the committed static build
generateBuildId: async () => "torchsiggui",
```

Effect:

- `npm run build` writes straight into the served folder.
- Repeated builds are byte-identical (verified by building twice and diffing recursively).
- The bundle was rebuilt after all the frontend source changes below.
- The first rebuild, made before any source edits, differed from the original committed bundle only in files that carry the build ID: the HTML, the `.txt` payload files, and the manifest folder name. All JS and CSS chunks were identical, which confirms the committed bundle matched the source at that point.
- The build also leaves an empty `torchsiggui/webbuild/_next/torchsiggui/` directory, which git ignores.

4.2 ESLint setup and frontend lint fixes

Files: `eslint.config.mjs` (new), `package.json`, `package-lock.json`, files under `src/`

Before: `"lint": "eslint"` existed, but ESLint was not installed and there was no configuration, so the script could not run.

New dev dependencies, and why these exact versions:

Package	Reason
<code>eslint ^9.39.5</code>	ESLint 10 was tried first, but <code>eslint-plugin-react</code> (bundled by <code>eslint-config-next</code>) crashes on it (<code>contextOrFilename.getFilename</code> is not a function). ESLint 9 is what Next 16 supports.
<code>eslint-config-next ^16.3.4</code>	Matches the installed <code>next</code> version and provides the <code>core-web-vitals</code> and <code>typescript</code> presets.
<code>"typescript": "npm:@typescript/typescript6@^6.0.2" and "@typescript/native": "npm:typescript@^7.0.2"</code>	<code>typescript-eslint</code> refuses to run with TypeScript 7.0, because TS 7 ships without a JS API. This is Microsoft's documented side-by-side setup: tools that <code>import "typescript"</code> get the TS 6 API, while <code>npx tsc</code> still runs TypeScript 7.0.2 (verified). Revert to plain <code>typescript@^7</code> once <code>typescript-eslint</code> supports TS 7.

`eslint.config.mjs`: a flat config that spreads `eslint-config-next/core-web-vitals` and `eslint-config-next/typescript`. It ignores `.next/`, `node_modules/`, `.venv/`, `torchsiggui/` (the built bundle), and `next-env.d.ts`. `.venv` had to be ignored because otherwise ESLint lints the JavaScript that matplotlib bundles inside the `virtualenv`.

Lint results on the existing code: 42 problems (31 errors, 11 warnings). All errors were fixed; 5 warnings remain.

Rule (count)	Location	Fix
no-explicit-any (5)	src/api/api-slice.ts	builder.mutation<any, any> → <unknown, unknown> for write-sample and write-dataset; <any, string> → <unknown, string> for cancel-dataset. No caller reads the mutation results, so behaviour is unchanged.
no-explicit-any (1)	src/components/form-parts/FormControls.tsx	Tooltip render prop typed as OverlayInjectedProps from react-bootstrap/Overlay.
no-explicit-any (16), plus 6 unused _state warnings	src/components/form-parts/select-style.ts	The style object became a typed generic function, customSelectStyle<IsMulti extends boolean>(): StylesConfig<FormOption, IsMulti>. The FormOption interface moved here from FormControls.tsx and is exported. Call sites use customSelectStyle<false>() for single selects and customSelectStyle<true>() for multi-selects. A single non-generic type does not type-check, because react-select's style callbacks are invariant in the option and multi types. Style values are unchanged.
prefer-const (7)	FormControls.tsx, DownloadList.tsx (x2), TransformParameters.tsx (x2), transform-slice.ts, transform-utils.ts	let → const via ESLint autofix, for variables that are never reassigned. No behaviour change.
react-hooks/refs (2)	src/providers/StoreProvider.tsx	The store was created lazily in a useRef that was read during render, which React 19's rules flag. Replaced with const [store] = useState(makeStore);. Passing the function rather than calling it makes React create the store exactly once per provider instance, the same behaviour as before. The unused AppStore import was removed.

Remaining warnings (they do not fail CI):

- FormControls.tsx (x2): unused _action parameters, required by the react-select callback signature.
- TransformAddModal.tsx: unused parent prop.

- `SpectrogramImage.tsx` (x2): an `<img>` without `alt` text, and Next.js's `no-img-element` suggestion. `next/image` offers little with a static export.

4.3 Ruff (Python lint) setup and fixes

Files: `pyproject.toml`, `torchsiggui/app.py`, `torchsiggui/main.py`, `torchsiggui/files/database_io.py`, `torchsiggui/utils/torchsig_interface.py`, `tests/test_downloads.py`, `tests/test_lifespan.py`

Configuration, chosen to match the existing code style so that no mass reformat is needed:

```
[tool.ruff]
line-length = 120
indent-width = 2
target-version = "py310"
extend-exclude = ["torchsiggui/webbuild", "node_modules"]

[tool.ruff.lint]
# Allows single-line if statements, which the codebase uses for short guards
ignore = ["E701"]

[tool.ruff.format]
quote-style = "single"
```

`ruff format` was **not** run on the codebase; only `ruff check` is enforced. `ruff` was added to the `dev` extra.

Findings fixed (19):

- `torchsiggui/app.py`:
 - Removed unused imports `MODULE_LOCK_FILE`, `WEBBUILD_FOLDER`, `get_worker_info`, `psutil`, `asynccontextmanager`, `FileLock`, `makedirs`, `getpid` and `StaticFiles`, left over from when the lifespan code lived in this file.
 - In the WebSocket loop, `msg = await websocket.receive_text()` → `await websocket.receive_text()`, since the value was never used.
- `torchsiggui/main.py`: removed the unused `MODULE_LOCK_FILE` and `FileLock` imports (see §3.3).
- `torchsiggui/files/database_io.py`:
 - Removed the unused `pathlib.Path` import.
 - `except Exception as e:` → `except Exception:` in three places. The handlers roll back and re-raise with a bare `raise`, so `e` was unused.
- `torchsiggui/utils/torchsig_interface.py`: `base_transform_map` is unused because the return statement that uses it is commented out. It was kept with `# noqa: F841` to preserve that intended future use.
- `tests/test_downloads.py`: `signal = reader.read(0)` → `reader.read(0)`; the test only checks that reading succeeds.
- `tests/test_lifespan.py`: `with TestClient(app) as client:` → `with TestClient(app):`.

4.4 WebSocket URL no longer hardcoded to localhost

File: `src/api/api-slice.ts`

Before:

```
const prodPort = (typeof window === "undefined") ? "8000" : window.location.port;
const wsPort = process.env.NODE_ENV === "development" ? "8000" : prodPort;
// ...
const ws = new WebSocket("ws://localhost:" + wsPort + "/ws");
```

After:

```
function buildWebSocketLink() {
  if (process.env.NODE_ENV === "development") {
    return "ws://localhost:8000/ws";
  }
  const protocol = window.location.protocol === "https:" ? "wss:" : "ws:";
  return protocol + "://" + window.location.host + "/ws";
}
// ...
const ws = new WebSocket(buildWebSocketLink());
```

Why:

- **Correct host:** in production the same server hosts the page and the WebSocket, so the page's own host is always the right address. The old code broke when the page was opened by a hostname other than `localhost`, or over `https`.
- `window` **is safe:** the function is only called inside `onCacheEntryAdded`, which runs in the browser, so `window` is always defined there. The old `typeof window` guard existed because that code ran when the module loaded during static prerendering.
- **Scope:** development mode behaviour is unchanged, and the production bundle no longer contains the string `localhost:8000` (verified).

4.5 Logging instead of `print`

File: `torchsiggui/app_write_spectrogram.py`

```
# Before
print('An error occured while trying to create a new spectrogram image: ' + str(error))

# After
import logging
logger = logging.getLogger(__name__)
logger.exception('An error occurred while trying to create a new spectrogram image: %s', error)
```

Errors now go through the logging system that uvicorn configures and include the full traceback. The typo “occured” is also fixed.

4.6 GitHub configuration (`.github/`)

Files (all new): `.github/workflows/ci.yml`, `.github/workflows/release.yml`,
`.github/ISSUE_TEMPLATE/bug_report.md`, `.github/ISSUE_TEMPLATE/feature_request.md`,
`.github/PULL_REQUEST_TEMPLATE.md`

`ci.yml` runs on pushes to `main` and on all pull requests. Its permissions are read-only, and a new push cancels any run still in progress for the same branch.

Job	Steps
api (matrix: Python 3.10, 3.12)	Checkout → setup-python with pip cache → install CPU-only <code>torch</code> and <code>torchaudio</code> from the PyTorch CPU index (the tests need no GPU, and this avoids several GB of CUDA wheels) → <code>pip install -e ".[dev]"</code> → <code>ruff check .</code> → <code>pytest</code> with <code>working-directory: tests</code> (the tests load data files by relative path).
interface (Node 22)	Checkout → setup-node with npm cache → <code>npm ci</code> → <code>npm run lint</code> → <code>npx tsc --noEmit</code> → <code>npm run build</code> → fail if <code>git status --porcelain torchsiggui/webbuild</code> is non-empty, with an error telling the contributor to rebuild and commit.

`release.yml` runs when a tag matching `v*` is pushed, with `contents: write` permission:

1. Build the interface (`npm ci`, `npm run build`) and fail if the committed bundle is stale.
2. Fail if the tag is not `v` followed by the `__version__` in `torchsiggui/__init__.py`.
3. Run `python -m build` to produce the wheel and sdist in `dist/`.
4. Create the GitHub Release with `softprops/action-gh-release@v2`, attaching `dist/*`, with auto-generated release notes. Tags containing a hyphen (e.g. `v0.0.1-rc1`) are marked as pre-releases.

Because of the version check, a test tag like `v0.0.1-rc1` fails unless `__version__` is temporarily set to `0.0.1-rc1`. Keep that in mind if you dry-run the workflow on a fork.

Issue templates:

- **Bug report** (label `bug`): description, reproduction steps, expected behaviour, screenshots/logs, and environment (TorchSigGUI, TorchSig, Python, OS, browser, GPU).
- **Feature request** (label `enhancement`): problem, proposed solution, alternatives.

PR template: a description, the linked issue, and a checklist that mirrors CI:

- `pytest` passes, run from `tests/`.
- `ruff check .` passes.
- `npm run lint` and `npx tsc --noEmit` pass.
- If `src/` changed, `torchsiggui/webbuild/` was rebuilt and committed.

4.7 `.gitattributes`

File: `.gitattributes` (new)

```
torchsiggui/webbuild/** linguist-generated=true -diff
package-lock.json linguist-generated=true
```

- **On GitHub:** generated files are collapsed in pull request diffs and excluded from the repository’s language statistics, which the minified JS would otherwise dominate.
- **Locally:** `-diff` makes `git diff` print “binary files differ” for the bundle instead of megabytes of minified text.
- **CI:** git still tracks the changes, so the check that the committed bundle is up to date keeps working.

5. README: local vs. remote usage

File: `README.md`, with new subsections under “Starting the Interface”.

Running on the Same Computer: run `torchsiggui` and open `http://localhost:8000`. If `--port` was used, substitute that port.

Running on a Remote Server, e.g. a GPU machine used from a laptop’s browser:

1. On the server: install and run `torchsiggui`. The README suggests `tmux` or `screen` to keep it running after you disconnect.
2. On the local computer: run `ssh -N -L 8000:localhost:8000 <user>@<server>` and keep that terminal open. The README explains the `-L <local port>:localhost:<server port>` format and that `-N` means no remote shell.
3. Open `http://localhost:8000` locally.

The section also covers:

- **Busy local port:** use a different one, e.g. `-L 9000:localhost:8000` and then `http://localhost:9000`.
- **Custom server port:** if the server was started with `--port`, use that value as the server-side port.
- **Where data lives:** datasets are stored on the remote server, and **Download** transfers them through the tunnel, so download before stopping the server.

Why a tunnel is required: uvicorn binds to `127.0.0.1` in `main.py`, so by design the server can’t be reached from the network, and there is no authentication.

Why a different local port works: this relies on the WebSocket change in §4.4, which makes the page connect back to whatever address it was loaded from. The old `localhost:<page port>` URL happened to work through a plain tunnel; the new one is also correct behind proxies and over `https`.

6. P2: Makefile, security policy, tests folder rename

6.1 Makefile

File: `Makefile` (new)

Modeled on TorchSig’s Makefile. Running `make` with no target prints help generated from the `##` comments. `PYTHON` and `NPM` can be overridden, e.g. `make test PYTHON=.venv/bin/python`.

Target	Action
<code>help</code>	List targets (default).
<code>install</code>	<code>install-api</code> + <code>install-web</code> .

Target	Action
<code>install-api</code>	<code>\$(PYTHON) -m pip install -e ".[dev]"</code>
<code>install-web</code>	<code>npm ci</code>
<code>dev-api</code>	<code>torchsiggui --dev</code>
<code>dev-web</code>	<code>npm run dev</code>
<code>build-web</code>	<code>npm run build</code> (into <code>torchsiggui/webbuild</code>)
<code>test</code>	<code>cd tests && \$(PYTHON) -m pytest</code>
<code>lint</code>	<code>lint-api</code> (<code>ruff check .</code>) + <code>lint-web</code> (<code>npm run lint</code> , <code>npx tsc --noEmit</code>).
<code>check</code>	Same checks as CI: lint, test, build, then fail if <code>torchsiggui/webbuild</code> has uncommitted changes (uses <code>git status --porcelain</code>).
<code>package</code>	Builds the web bundle, then <code>python -m build</code> into <code>dist/</code> .
<code>clean</code>	Removes <code>build/</code> , <code>dist/</code> , <code>*.egg-info</code> , <code>.next</code> , the <code>pytest</code> and <code>ruff</code> caches, coverage and JUnit reports (in the root and <code>tests/</code>), <code>tsconfig.tsbuildinfo</code> , and <code>__pycache__</code> folders outside <code>node_modules</code> and <code>.venv</code> .

`make test` uses the first `python` on `PATH`, so activate the virtualenv first (the local `.envrc` does this through `direnv`).

6.2 SECURITY.md

File: `SECURITY.md` (new)

- **Supported versions:** only the latest release (0.0.x) receives security fixes.
- **Security model:**
 - There is no authentication.
 - The server binds to loopback only.
 - Don't expose it publicly, or behind a reverse proxy or port forward, without adding authentication.
 - On shared multi-user machines, any local user can connect to `127.0.0.1`.
 - Dataset generation can use large amounts of disk, memory, CPU and GPU.
 - Data is temporary and deleted when the server stops.
 - It links to the README's remote-server section.
- **Reporting:** don't use public issues. Report through GitHub private vulnerability reporting at <https://github.com/TorchDSP/torchsig-gui/security/advisories/new>, including impact, reproduction steps, and affected versions.

Action required: private vulnerability reporting must be enabled in the GitHub repository (Settings → Code security → Private vulnerability reporting), or the link in `SECURITY.md` won't work.

6.3 Rename `testing/` → `tests/`

Changes: folder move, plus references in `pyproject.toml`, `README.md`, `.github/workflows/ci.yml`, and `.github/PULL_REQUEST_TEMPLATE.md`

- **Why:** matches TorchSig and common Python convention.
- **How:** the move used `mv`, not `git mv`. Git normally detects the rename automatically once both sides are staged (`git add -A`).
- **References updated:** `testpaths`, the README testing section, the CI `working-directory` and its comment, and the PR checklist. No other file referenced the old name.
- **Test files:** contents are unchanged apart from the two Ruff fixes in §4.3.

The tests still open data with relative paths like `./test_data/data_default.json`, so pytest must be run **from inside** `tests/`. Running from the repository root gives 9 `FileNotFoundError` failures. This was already the case in the original code (verified).

7. Verification performed

Check	Result	Details
Baseline pytest before any change (from <code>testing/</code>)	16 passed	Plus 29 subtests. Establishes that any later failures would be regressions.
pytest after P0 changes	16 passed	Coverage now reported for <code>torchsiggui</code> (85% total at that point).
pytest after P1 changes	16 passed	29 subtests passed.
pytest after the P2 rename (<code>make test</code>)	16 passed	Run from <code>tests/</code> via the Makefile.
Wheel build (<code>pip wheel . --no-deps</code>)	Pass	<code>torchsiggui-0.0.1-py3-none-any.whl</code> , 48 files, including <code>torchsiggui/files/database_sql.sql</code> , <code>torchsiggui/webbuild/index.html</code> and all <code>_next/static</code> chunks. Metadata shows <code>Version: 0.0.1</code> , <code>License-Expression: MIT</code> , the project URLs, <code>torchsig==2.2.0</code> and <code>fastapi>=0.141</code> .
Non-editable smoke test: server run from the unpacked wheel, outside the repo, with <code>TORCHSIGGUI_DATA_DIR</code> set to a scratch folder	Pass	<code>GET / → 200</code> . <code>GET /api/metadata-defaults → 200</code> with JSON. <code>datasets/state.db</code> was created in the chosen data folder, and the folder was removed on shutdown (SIGINT).

Check	Result	Details
Live server after the frontend rebuild, using the installed <code>torchsiggui</code> command	Pass	<code>GET /</code> → 200. WebSocket <code>/ws</code> accepted with the connection open. No message arrived within 5 s, which is expected because the feeds only push when sample or file state changes.
<code>ruff check .</code>	Pass	“All checks passed!”
<code>npm run lint</code>	Pass	0 errors, 5 warnings (listed in §4.2).
<code>npx tsc --noEmit</code> (TypeScript 7.0.2)	Pass	No type errors.
<code>npm run build</code> reproducibility	Pass	Two consecutive builds diffed recursively: identical.
Hardcoded URLs in the production bundle	Pass	<code>localhost:8000</code> no longer appears in <code>torchsiggui/webbuild</code> .
Workflow YAML syntax	Pass	Both workflow files parse with PyYAML.
<code>make help</code> , <code>make -n check</code> , <code>make lint-api</code>	Pass	Targets list correctly; <code>check</code> expands to the expected commands.

Not verified:

- The GitHub Actions workflows have not run on GitHub, because nothing was pushed.
- The WebSocket URL change was tested with a Python WebSocket client against a live server, not in a real browser and not through an actual SSH tunnel.
- That a Linux build produces the same chunk hashes as the macOS build (see §9).
- Full end-to-end dataset generation and download through the UI. The API tests cover it, but nobody clicked through it manually.
- Python 3.10 and 3.12 specifically; local testing used Python 3.14.

8. Complete file inventory

Modified

File	Change (section)
<code>torchsiggui/__init__.py</code>	Version <code>0.0.1</code> (3.1)
<code>package.json</code>	Version, <code>engines</code> , build output path, ESLint and TypeScript dev dependencies (3.1, 3.5, 4.1, 4.2)
<code>package-lock.json</code>	Version; lock entries for the new dev dependencies (3.1, 4.2)
<code>pyproject.toml</code>	Metadata, dependencies, dev extra, package data, URLs, coverage, ruff, testpaths (3.2, 4.3, 6.3)

File	Change (section)
<code>LICENSE</code>	Copyright line (3.5)
<code>README.md</code>	Install, prerequisites, config, data folder, local/remote usage, dev setup, tests path (3.5, 5, 6.3)
<code>next.config.ts</code>	Fixed <code>generateBuildId</code> (4.1)
<code>torchsiggui/main.py</code>	<code>remove_crashed_workers()</code> ; unused imports removed (3.3, 4.3)
<code>torchsiggui/files/file_io.py</code>	Data folder in the user cache (3.4)
<code>torchsiggui/app.py</code>	Unused imports and variable removed (4.3)
<code>torchsiggui/app_write_spectrogram.py</code>	Logging (4.5)
<code>torchsiggui/files/database_io.py</code>	Unused import and exception variables removed (4.3)
<code>torchsiggui/utils/torchsig_interface.py</code>	<code># noqa: F841</code> (4.3)
<code>src/api/api-slice.ts</code>	WebSocket URL; <code>unknown</code> mutation types (4.2, 4.4)
<code>src/components/form-parts/select-style.ts</code>	Typed generic style function; exports <code>FormOption</code> (4.2)
<code>src/components/form-parts/FormControls.tsx</code>	Tooltip prop type; imports <code>FormOption</code> ; explicit style generics; <code>const</code> (4.2)
<code>src/providers/StoreProvider.tsx</code>	<code>useState(makeStore)</code> (4.2)
<code>src/features/spectrogram/DownloadList.tsx</code>	<code>const</code> (4.2)
<code>src/features/transform-form/TransformParameters.tsx</code>	<code>const</code> (4.2)
<code>src/features/transform-form/transform-slice.ts</code>	<code>const</code> (4.2)
<code>src/features/transform-form/transform-utils.ts</code>	<code>const</code> (4.2)
<code>tests/test_downloads.py</code> , <code>tests/test_lifespan.py</code>	Unused variables (4.3)
<code>torchsiggui/webbuild/**</code>	Regenerated from the current source with the fixed build ID. The old <code>_next/static/tMHix5hQb_ydNult2RoH/</code> folder is replaced by <code>_next/static/torchsiggui/</code> (4.1)

Added

File	Purpose (section)
<code>.env.example</code>	Example server configuration (3.5)
<code>eslint.config.mjs</code>	ESLint flat config (4.2)
<code>.gitattributes</code>	Marks generated files (4.7)
<code>.github/workflows/ci.yml</code>	Continuous integration (4.6)
<code>.github/workflows/release.yml</code>	Tag-triggered GitHub Release (4.6)

File	Purpose (section)
<code>.github/ISSUE_TEMPLATE/bug_report.md</code>	Issue template (4.6)
<code>.github/ISSUE_TEMPLATE/feature_request.md</code>	Issue template (4.6)
<code>.github/PULL_REQUEST_TEMPLATE.md</code>	PR checklist (4.6)
<code>Makefile</code>	Developer tasks (6.1)
<code>SECURITY.md</code>	Security policy (6.2)
<code>changed.md</code> , <code>changed.pdf</code>	This report and its source. Delete them or leave them uncommitted, as you prefer.

Renamed

From	To
<code>testing/</code>	<code>tests/</code> (all 7 Python files and <code>test_data/</code> ; see 6.3)

Local-only artifacts (git-ignored, not part of the change)

- **Still present:** `node_modules/` (from `npm ci` / `npm install`), `tsconfig.tsbuildinfo`, `.ruff_cache/`, `.pytest_cache/`, `.coverage`, `torchsiggui.egg-info/`.
- **Removed after use:** generated `coverage.xml` and `report.xml` files, the temporary `./webbuild` and `.next` build folders, and `__pycache__` in the tests folder.
- **Outside the repository:** test wheels and smoke-test data were written to a temporary scratch directory.

9. Items needing your attention

#	Item	Details and recommendation
1	Enable private vulnerability reporting	Needed for the reporting link in <code>SECURITY.md</code> to work. GitHub → Settings → Code security.
2	Third-party license files	<code>NOTICE</code> , <code>THIRD_PARTY_LICENSES</code> and <code>licenses/</code> still list Python and test dependencies that are not redistributed, while the bundled JS <i>does</i> redistribute npm packages. Recommendation: regenerate <code>THIRD_PARTY_LICENSES</code> from the production npm dependencies (e.g. with <code>license-checker --production</code>) and drop the Python entries, or remove the files if legal review agrees.

#	Item	Details and recommendation
3	Cross-platform build hashes	The CI check that the committed bundle is up to date assumes Linux and macOS produce identical bundles. If the first CI run reports <code>torchsiggui/webbuild</code> as out of date with no source changes, commit the bundle built on Linux (e.g. from a CI artifact or a Linux container).
4	First CI run	The workflows are untested on GitHub. Watch the first run, especially the CPU-only PyTorch install and TorchSig 2.2.0 dependency resolution on Python 3.10 and 3.12.
5	npm audit	<code>npm ci</code> reported known vulnerabilities in the dependency tree. <code>npm audit fix</code> was deliberately not run. Review <code>npm audit</code> before release.
6	TypeScript alias workaround	The <code>typescript</code> → TS 6 alias exists only for typescript-eslint. Remove it once typescript-eslint supports TS 7.
7	<code>httpx2</code>	Confirm this is the intended test dependency, rather than <code>httpx</code> .
8	Browser and tunnel test	Before tagging, click through the UI once locally (Generate Sample, Generate Dataset, Download, Cancel), and once through <code>ssh -L</code> with a different local port.
9	Data folder change is user-visible	Developers used to finding <code>datasets/</code> in the repository root will now find it under <code>~/.cache/torchsiggui/</code> . This is documented in the README, and <code>TORCHSIGGUI_DATA_DIR</code> can point it somewhere else.
10	Tests require <code>cd tests</code>	Optional follow-up: build the test data paths from <code>Path(__file__).parent</code> so <code>pytest</code> works from the repo root, then simplify CI and the Makefile.
11	Remaining lint warnings	5 ESLint warnings (§4.2). Adding <code>alt</code> text to the <code></code> is a small accessibility fix worth doing.

#	Item	Details and recommendation
12	Skipped by choice	<code>CHANGELOG.md</code> , <code>CONTRIBUTING.md</code> , Dockerfile, coverage upload, README badges, screenshot and citation. With no changelog, the release workflow's auto-generated notes serve as the release notes.
13	Unused locking code	The commented-out <code>FileLock</code> blocks in <code>main.py</code> would need their imports restored if multi-worker mode is enabled.

10. Suggested commit and release procedure

1. Review the working-tree changes (`git status` , `git diff`). `.gitattributes` hides the webbuild diffs, so use `git diff --text` if you want to see them.
2. Run `make install` if needed, then run `make check` locally.
3. Stage everything, including the rename, with `git add -A` , and commit. Optionally split the work into logical commits: packaging/P0, lint and CI, docs, rename.
4. Push to a branch and open a PR to trigger `ci.yml` . Resolve any Linux build-hash mismatch (§9, item 3).
5. Merge to `main` , and enable private vulnerability reporting.
6. Tag and push: `git tag -a v0.0.1 -m "TorchSigGUI 0.0.1"` , then `git push origin v0.0.1` .
7. `release.yml` checks the bundle and version, builds the wheel and sdist, and publishes the GitHub Release. Edit the auto-generated notes if you want.
8. Smoke-test the published install: `pip install git+https://github.com/TorchDSP/torchsig-gui.git@v0.0.1` , then `torchsiggui --version` (expect `TorchSigGUI 0.0.1`).